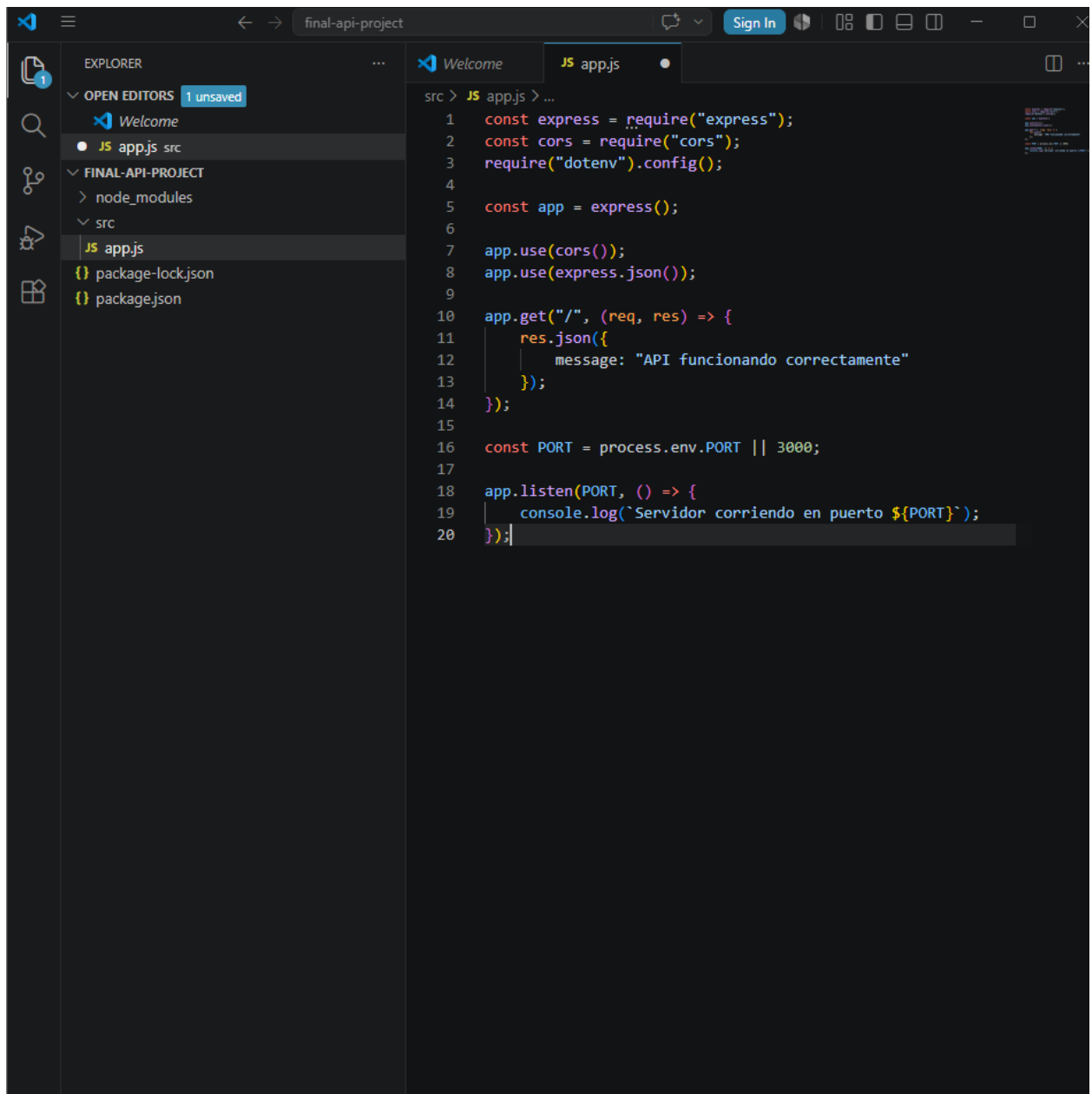
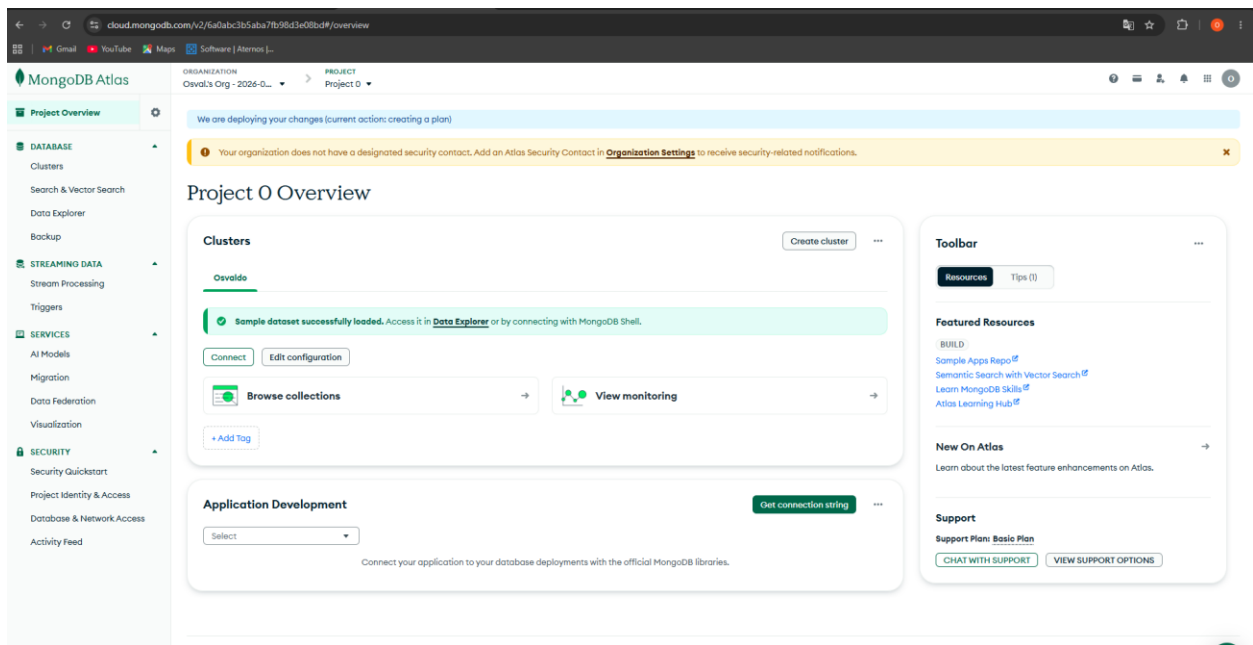
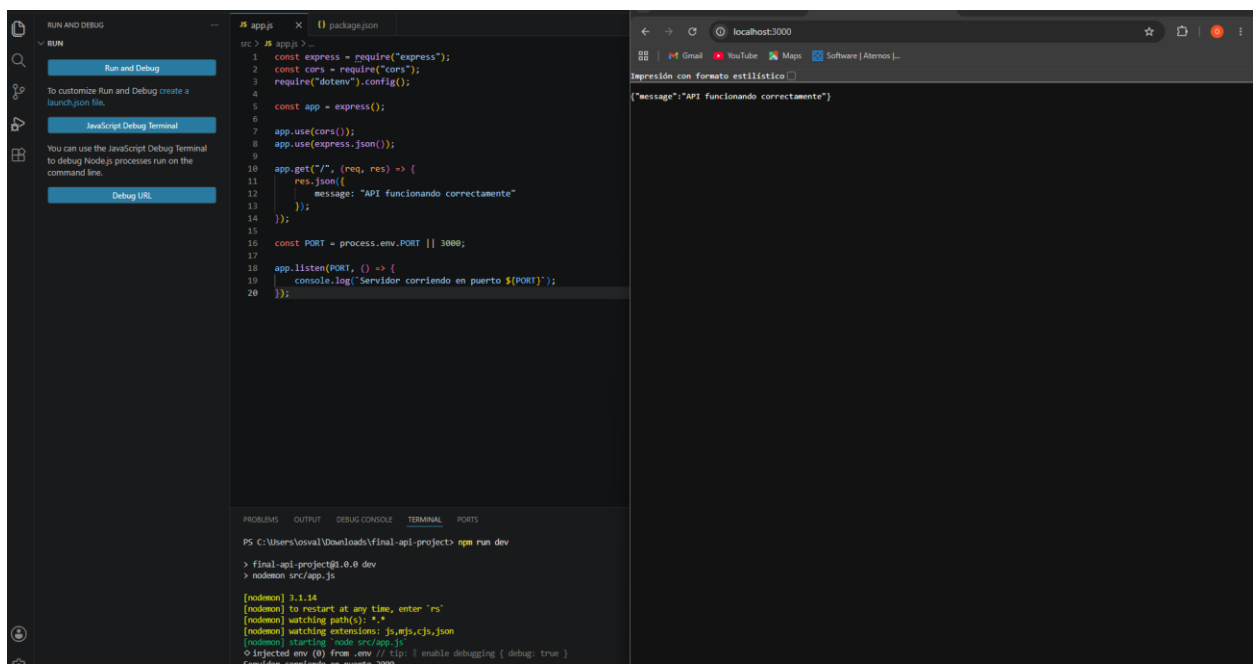
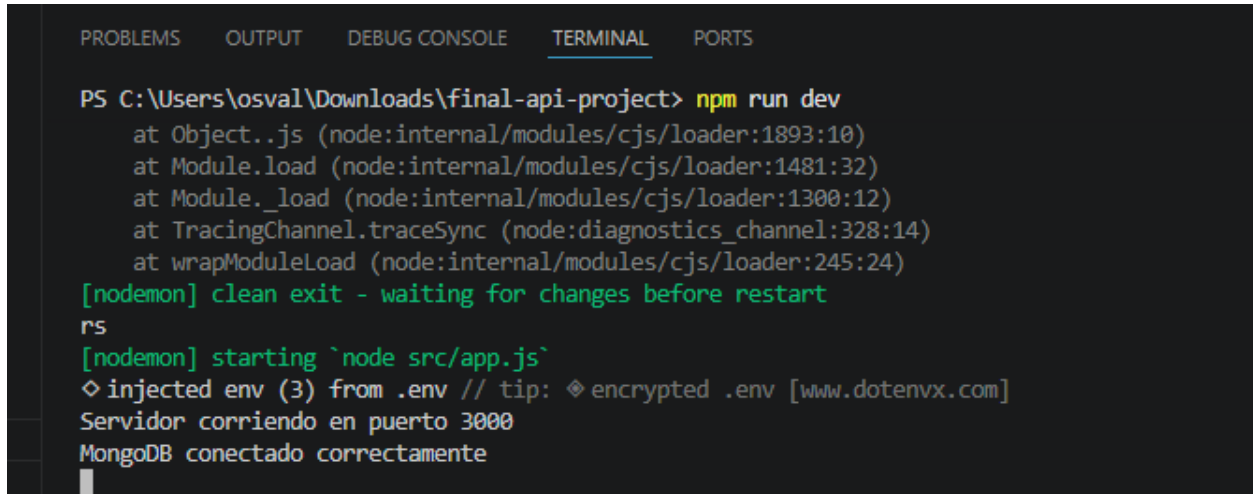
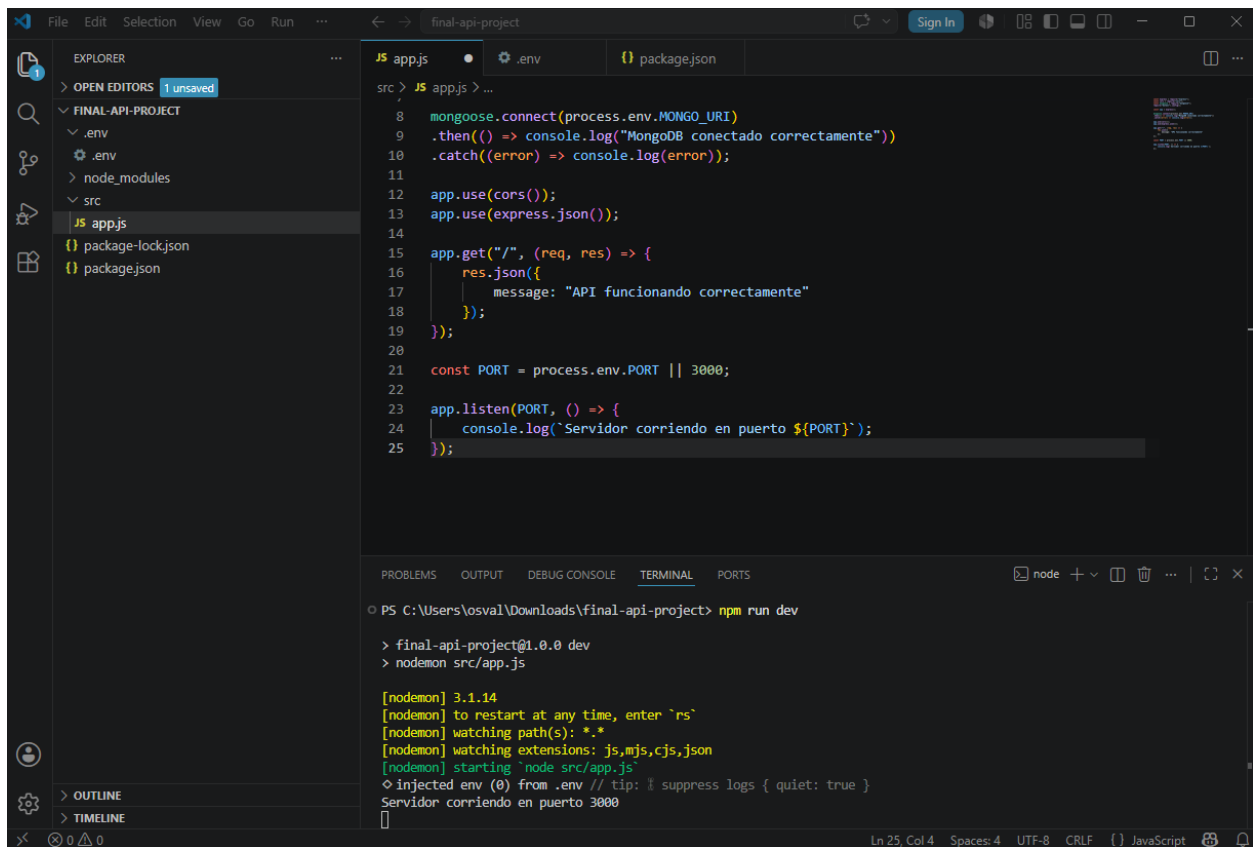








JS app.js

.env

JS User.js

JS Task.js

packag...  ...

src > models > JS User.js > ...

```
1  const mongoose = require("mongoose");
2
3  const userSchema = new mongoose.Schema({
4    email: {
5      type: String,
6      required: true,
7      unique: true
8    },
9
10   password: {
11     type: String,
12     required: true
13   }
14 }, {
15   timestamps: true
16 });
17
18
19 module.exports = mongoose.model("User", userSchema);
```

```
src > models > JS Task.js > ...
1  const mongoose = require("mongoose");
2
3  const taskSchema = new mongoose.Schema({
4
5      title: {
6          type: String,
7          required: true
8      },
9
10     description: {
11         type: String
12     },
13
14     status: {
15         type: String,
16         enum: ["pending", "in_progress", "done"],
17         default: "pending"
18     },
19
20     dueDate: {
21         type: Date
22     },
23
24     user: {
25         type: mongoose.Schema.Types.ObjectId,
26         ref: "User"
27     }
28 }, {
29     timestamps: true
30 });
31
32
33 module.exports = mongoose.model("Task", taskSchema);
34
35
```

```
src > routes > JS authRoutes.js > ...
1  const express = require("express");
2
3  const router = express.Router();
4
5  router.get("/", (req, res) => {
6      res.json({
7          message: "Rutas auth funcionando"
8      });
9  });
10
11 module.exports = router;
```

rc > JS app.js > ...

```
1  const authRoutes = require("./routes/authRoutes");
2  const express = require("express");
3  const cors = require("cors");
4  const mongoose = require("mongoose");
5  require("dotenv").config();
6
7  const app = express();
8
9  mongoose.connect(process.env.MONGO_URI)
10 .then(() => console.log("MongoDB conectado correctamente"))
11 .catch((error) => console.log(error));
12
13 app.use(cors());
14 app.use(express.json());
15 app.use("/api/auth", authRoutes);
16
17 app.get("/", (req, res) => {
18   res.json({
19     message: "API funcionando correctamente"
20   });
21 });
22
23 const PORT = process.env.PORT || 3000;
24
25 app.listen(PORT, () => {
26   console.log(`Servidor corriendo en puerto ${PORT}`);
27 });
28
```

The image shows a VS Code editor with a project named 'FINAL-API-PROJECT'. The file explorer on the left shows the project structure, including 'node_modules', 'src', 'config', 'controllers', 'middlewares', 'models', 'tasks', 'users', 'routes', and 'authRoutes.js'. The main editor displays the 'app.js' file, which is a Node.js application using Express, MongoDB, and CORS. The code is as follows:

```
1 const authRoutes = require("../routes/authRoutes");
2 const express = require("express");
3 const cors = require("cors");
4 const mongoose = require("mongoose");
5 require("dotenv").config();
6
7 const app = express();
8
9 mongoose.connect(process.env.MONGO_URI)
10 .then(() => console.log("MongoDB conectado correctamente"))
11 .catch((error) => console.log(error));
12
13 app.use(cors());
14 app.use(express.json());
15 app.use("/api/auth", authRoutes);
16
17 app.get("/", (req, res) => {
18   res.json({
19     message: "API funcionando correctamente"
20   });
21 });
22
23 const PORT = process.env.PORT || 3000;
24
25 app.listen(PORT, () => {
26   console.log(`Servidor corriendo en puerto ${PORT}`);
27 });
```

The terminal at the bottom shows the command 'npm run dev' being executed, which starts the application on port 3000. The output indicates that the application is running successfully, with MongoDB connected and the server listening on port 3000.

The browser window on the right shows the URL 'localhost:3000/api/auth'. The response is a 404 Not Found error, indicating that the API endpoint is not found.

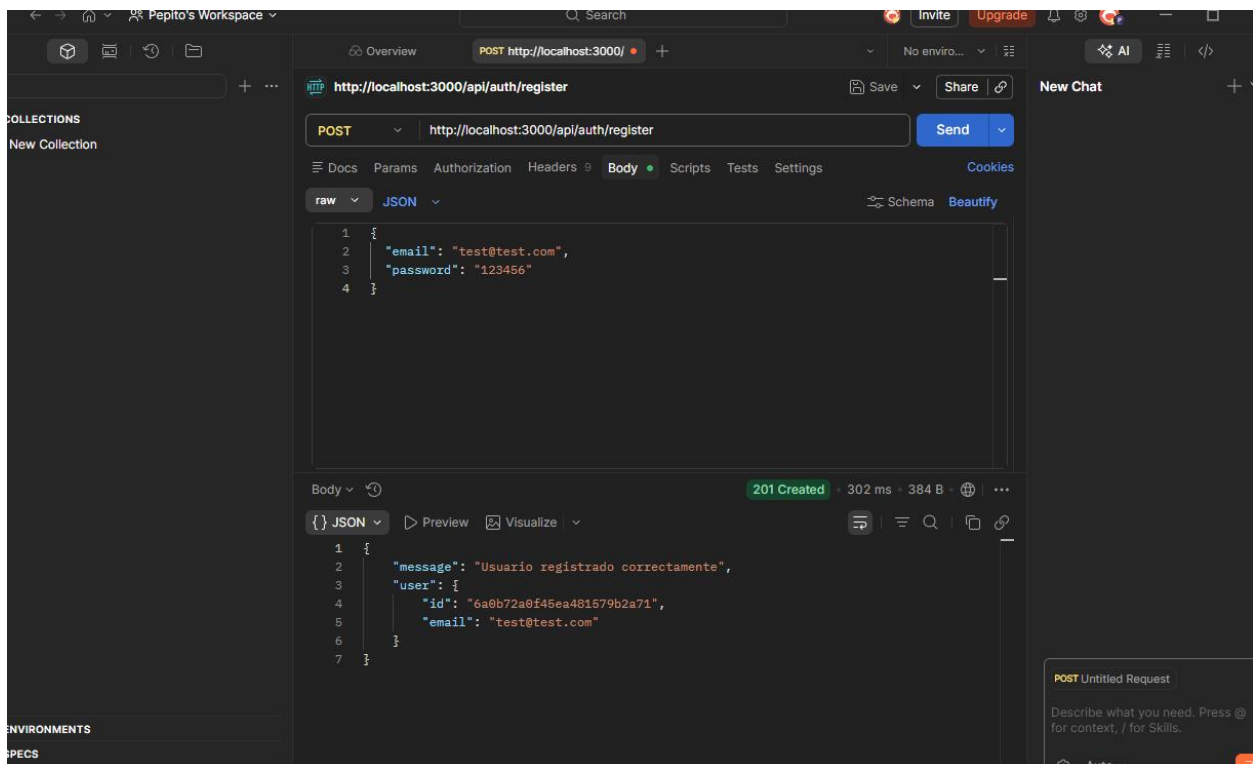
src > routes > JS authRoutes.js > ...

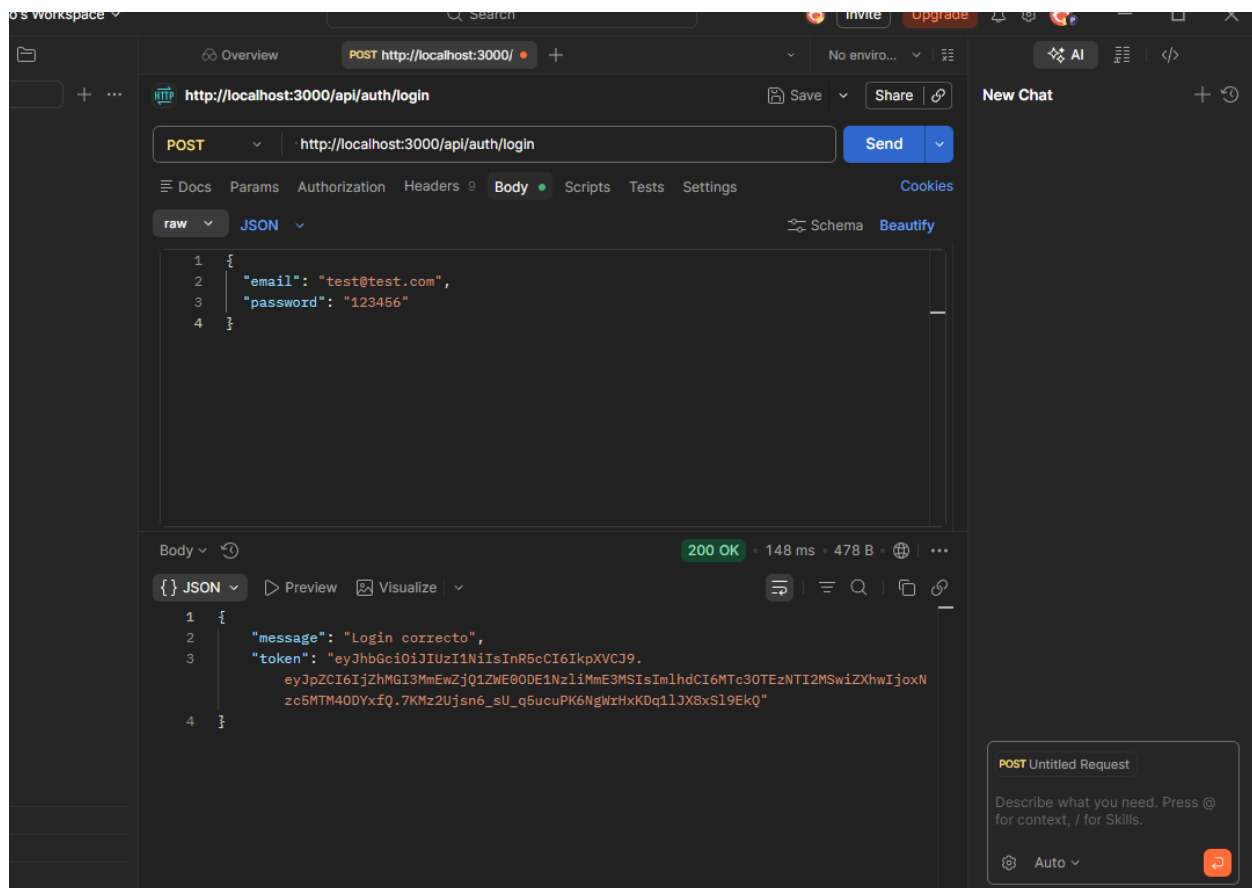
```
6
7   router.post("/register", async (req, res) => {
8     try {
9       const { email, password } = req.body;
10
11       if (!email || !password) {
12         return res.status(400).json({
13           error: true,
14           message: "Email y password son requeridos"
15         });
16       }
17
18       const userExists = await User.findOne({ email });
19
20       if (userExists) {
21         return res.status(400).json({
22           error: true,
23           message: "El usuario ya existe"
24         });
25       }
26
27       const hashedPassword = await bcrypt.hash(password, 10);
28
29       const user = await User.create({
30         email,
31         password: hashedPassword
32       });
33
34       res.status(201).json({
35         message: "Usuario registrado correctamente",
36         user: {
37           id: user._id,
38           email: user.email
39         }
40       });
41
42     } catch (error) {
```



```
> OPEN EDITORS
FINAL-API-PROJECT
  > node_modules
  > src
    > config
    > controllers
    > midlewares
  > models
  JS Task.js
  JS User.js
  > routes
    JS authRoutes.js
  JS app.js
  .env
  package-lock.json
  package.json

src > routes > JS authRoutes.js > ...
7 router.post("/register", async (req, res) => {
41
42   } catch (error) {
43     res.status(500).json({
44       error: true,
45       message: "Error al registrar usuario"
46     });
47   }
48 });
49
50 module.exports = router;
```





final-api-project

EXPLORER

1 unsaved

FINAL-API-PROJECT

- node_modules
- src
 - config
 - controllers
 - middlewares
 - JS authMiddleware.js**
 - models
 - JS Task.js
 - JS User.js
 - routes
 - JS authRoutes.js
- app.js
- .env
- package-lock.json
- package.json

src > middlewares > JS authMiddleware.js > ...

```
1 const jwt = require("jsonwebtoken");
2
3 const authMiddleware = (req, res, next) => {
4   const authHeader = req.headers.authorization;
5
6   if (!authHeader) {
7     return res.status(401).json({
8       error: true,
9       message: "No hay token"
10    });
11  }
12
13  const token = authHeader.split(" ")[1];
14
15  try {
16    const decoded = jwt.verify(token, process.env.JWT_SECRET);
17    req.user = decoded;
18    next();
19  } catch (error) {
20    return res.status(401).json({
21      error: true,
22      message: "Token inválido"
23    });
24  }
25 };
26
27 module.exports = authMiddleware;
```

EXPLORER

2 unsaved

FINAL-API-PROJECT

- node_modules
- src
 - config
 - controllers
 - middlewares
 - JS authMiddleware.js
 - models
 - JS Task.js
 - JS User.js
 - routes
 - JS authRoutes.js
 - JS taskRoutes.js**
- app.js
- .env
- package-lock.json
- package.json

src > routes > JS taskRoutes.js > ...

```
1 const express = require("express");
2 const authMiddleware = require("../middlewares/authMiddleware");
3
4 const router = express.Router();
5
6 router.get("/", authMiddleware, (req, res) => {
7   res.json({
8     message: "Rutas de tareas protegidas funcionando",
9     user: req.user
10  });
11 });
12
13 module.exports = router;
```

src > middlewares > JS authMiddleware.js > ...

```
1  const jwt = require("jsonwebtoken");
2
3  const authMiddleware = (req, res, next) => {
4    const authHeader = req.headers.authorization;
5
6    if (!authHeader) {
7      return res.status(401).json({
8        error: true,
9        message: "No hay token"
10     });
11   }
12
13   const token = authHeader.split(" ")[1];
14
15   try {
16     const decoded = jwt.verify(token, process.env.JWT_SECRET);
17     req.user = decoded;
18     next();
19   } catch (error) {
20     return res.status(401).json({
21       error: true,
22       message: "Token inválido"
23     });
24   }
25 };
26
27 module.exports = authMiddleware;
```

⌵

←

→

🏠

👤 Pepito's Workspace

🔍 Search

🔥 Invite

📈 Upgrade

🔔

⚙️

🌐

—

□

📁

📄

🕒

📁

⌵

+

...

🔗

Overview

POST http://localhost:3000/

+

+

No enviro...

⌵

⌵

🔗 AI

⌵

⌵

⌵

+

🔗

http://localhost:3000/api/tasks

📁 Save

🔗 Share

📄 New Chat

+

POST

⌵

http://localhost:3000/api/tasks

Send

⌵

☰ Docs

Params

Authorization

•

Headers 10

Body •

Scripts

Tests

Settings

Cookies

raw

⌵

JSON

⌵

🔗 Schema

Beautify

1

2

3

4

5

}

Body

⌵

🔗

201 Created

• 90 ms • 505 B •

🌐

⌵

{}

JSON

⌵

▶ Preview

🖼️ Visualize

⌵

1

2

3

4

5

6

7

8

9

10

}

⌵ ENVIRONMENTS

⌵ SPECS

⌵ FLOWS

🔗 Connect Git

📄 Console

📄 Terminal

Globals

Vault

Tools

⌵

⌵

⌵

POST http://localhost:3000/api/tasks

Describe what you need. Press @ for context, / for Skills.

⚙️ Auto

+

+

...

+

☆

...

HTTP

http://localhost:3000/api/tasks/6a0b7c6b5cd70544703de792

Save

Share

PUT

http://localhost:3000/api/tasks/6a0b7c6b5cd70544703de792

Send

Docs

Params

Authorization

Headers 10

Body

Scripts

Tests

Settings

Cookies

raw

JSON

Schema

Beautify

```
1 {
2   "status": "done"
3 }
```

Body

200 OK · 170 ms · 497 B ·

{}

JSON

Preview

Visualize

```
1 {
2   "_id": "6a0b7c6b5cd70544703de792",
3   "title": "Proyecto Final",
4   "description": "Terminar API REST",
5   "status": "done",
6   "user": "6a0b72a0f45ea481579b2a71",
7   "createdAt": "2026-05-18T20:54:03.646Z",
8   "updatedAt": "2026-05-18T20:58:45.825Z",
9   "__v": 0
10 }
```

Overview

POST http://

POST http://

PUT http://

DEL http://

GET http://

+

No enviro...

⋮

⚙️

+

...

🔗

http://localhost:3000/api/tasks/6a0b7c6b5cd70544703de792

📄 Save

🔗 Share

New Chat

+

↺

DELETE

⌵

http://localhost:3000/api/tasks/6a0b7c6b5cd70544703de792

Send

⌵

🔑 Docs

🔑 Params

🔑 Authorization

🔑 Headers 8

🔑 Body

🔑 Scripts

🔑 Tests

🔑 Settings

🔑 Cookies

Auth Type

Bearer Token

⌵

Token

.....👁🔒

The authorization header will be automatically generated when you send the request. Learn more about [Bearer Token](#) authorization.

Body

⌵

🕒

200 OK

73 ms

310 B

🌐

⋮

{ } JSON

▶ Preview

🖨 Visualize

⌵

1 {

2 |

3 }

"message": "Tarea eliminada correctamente"

DEL http://localhost:3...

Describe what you need. Press @ for context, / for Skills.